

# Gen-AI training pre-requisites and pre-read

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Overview .....                                                             | 1  |
| Install Python 3.12.10 .....                                               | 2  |
| Install and setup VS Code .....                                            | 2  |
| Understanding and creating a Virtual Environment for Gen AI projects ..... | 3  |
| Understanding Project Root folder .....                                    | 3  |
| Create a virtual environment for your project .....                        | 3  |
| Install packages needed for Gen AI development .....                       | 4  |
| Python essentials .....                                                    | 8  |
| Using functions in python .....                                            | 8  |
| Working with data tables using pandas .....                                | 8  |
| Using .env for API keys and private configuration .....                    | 8  |
| Working with data files .....                                              | 9  |
| Using matplotlib for data visualization and analysis .....                 | 9  |
| Accessing and consuming services using APIs .....                          | 9  |
| Google Colab and Kaggle .....                                              | 9  |
| Create your first colab notebook .....                                     | 10 |
| Working with secret keys in Colab .....                                    | 11 |
| Working with data files in Colab .....                                     | 13 |
| Git (Optional) .....                                                       | 16 |
| 1. Download Git .....                                                      | 16 |
| 2. Install Git .....                                                       | 16 |
| 3. Configure Git .....                                                     | 16 |
| 4. Verify Installation .....                                               | 16 |
| Troubleshooting .....                                                      | 17 |

## Overview

This document describes the setup needed to build Gen AI Apps and also gives a primer on Python to get you ready for the Gen AI training.

It is expected to take 2 to 4 hours to go through the below steps depending on your existing familiarity with these topics.

## Install Python 3.12.10

### 1. Install Python

Install **3.12.10** version of Python from:

**DO NOT INSTALL LATEST (3.13.x) PYTHON VERSION**

**When installing make sure the “Add python to system path” checkbox is selected**

Download link:

<https://www.python.org/downloads/release/python-31210/>

<https://www.python.org/downloads/release/python-31210/>

**Use Python 3.12.10 only as langchain and other AI python libraries do not work well with 3.13**

### 2. Test python

Open a command prompt and give the following command:

`python --version`

It should show the python version, namely, 3.12.7

*If it errors finding the executable, add python.exe location to System environment variable: PATH*

*You can find the location of python.exe by opening the windows menu, right click on Python app (open file location), copy path*

## Install and setup VS Code

### 1. Download and install [VS Code](https://code.visualstudio.com/)

<https://code.visualstudio.com/>

### 2. Install python extensions in VS Code

1. Open VS Code
2. Install the Python extension:
  - Click the Extensions icon in the sidebar (or press Ctrl+Shift+X)

- Search for "Python"
- Install the Microsoft Python extension

## Understanding and creating a Virtual Environment for Gen AI projects

### Understanding Project Root folder

Project root folder (such as **genaitraining**) or any folder in which you are putting your code, provides the base folder to work against.

When opening VS Code, this is the folder you will open (not any other subfolder)

This is important as your project virtual environment, private configuration (.env) will all reside in your project root folder.

Hence, whenever starting VS Code, always open the project root folder and not any other folder

All the code zips provided will need to be unzipped into a subfolder inside this folder

So you will eventually have a folder structure in VS Code like below:

#### **genaitraining**

|\_\_ .env

|\_\_ venv

|\_\_ eda

|\_\_ ml

..

### Create a virtual environment for your project

#### **What is Virtual Environment?**

Virtual environment facilitates creation of a separate folder where all your python dependencies are saved. This ensures that you do not have to reinstall any library everytime and also that the right versions of your libraries are available to your application code (no conflicts with any other project or setup on your machine)

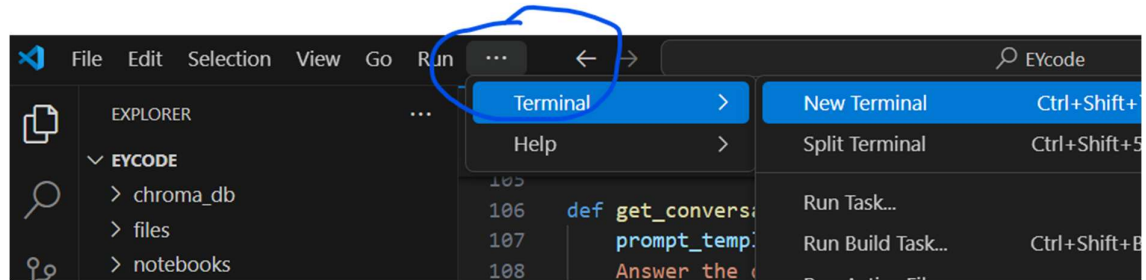
Following video explains the concept and benefits of virtual environments:

<https://youtu.be/KxvKCSwlUv8?si=6HQHCEJ2wkXnJ0QX>

Follow along the video below or steps outlined below to create a virtual environment

<https://youtu.be/GZbeL5AcTgw?si=cvCzwnbiXPZEWFMA>

1. Create a new folder for your project, called **genaitraining (or code)** in your home directory or in Desktop
2. Open the folder in VS Code (File > Open Folder)
3. Start a terminal



4. In the terminal window, give the following commands to create a new virtual environment:

```
python -m venv venv
```

```
venv\Scripts\activate
```

You will see a popup asking you to use this virtual environment for your project. Click on “Yes”

## Install packages needed for Gen AI development

Python package manager **pip** allows you to install all needed packages from a file commonly named as **requirements.txt**

We will create requirements.txt and put all the packages we need for Gen AI development in it. We will then install those packages in our virtual environment.

1. In VS Code, with the project root folder (genaitraining or code) selected, click on the +File icon to create a new file, name it “requirements.txt”
2. Put following content in requirements.txt and save the file (ctrl+s):

```
openai
python-dotenv
requests
vertexai
google-cloud-aiplatform
langchain-openai
langchain-community
langchain-text-splitters
chromadb
beautifulsoup4
```

pypdf  
wikipedia  
plotly  
tqdm  
datasets  
streamlit  
dotted-dict  
PyPDF2  
matplotlib  
seaborn  
torch  
transformers  
tensorflow  
tf-keras  
jupyter  
ipykernel

3. Install packages from requirements.txt:

**Make sure your virtual environment is active (you should see the venv name in the command prompt)**

```
14 "AZURE_OPENAI_ENDPOINT": os.getenv("AZURE_OPENAI_ENDPOINT"),
15 "AZURE_OPENAI_MODEL_DEPLOYMENT_NAME": os.getenv("AZURE_OPENAI_MODEL_DEPLO
16 "AZURE_OPENAI_MODEL_NAME": os.getenv("AZURE_OPENAI_MODEL_NAME"),
17 "AZURE_OPENAI_API_KEY": os.getenv("AZURE_OPENAI_API_KEY"),
18 "AZURE_OPENAI_API_VERSION": os.getenv("AZURE_OPENAI_API_VERSION")

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Downloading certifi-2024.8.30-py3-none-any.whl (167 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading h11-0.14.0-py3-none-any.whl (58 kB)
Installing collected packages: typing-extensions, sniffio, jiter, idna, h11, distro, co
ydantic-core, httpcore, anyio, pydantic, httpx, openai
Successfully installed annotated-types-0.7.0 anyio-4.6.0 certifi-2024.8.30 colorama-0.4
httpx-0.27.2 idna-3.10 jiter-0.6.1 openai-1.51.2 pydantic-2.9.2 pydantic-core-2.23.4 s
s-4.12.2
PS C:\Users\BN743PF\Downloads\EYcode\EYcode> python -m venv venv
PS C:\Users\BN743PF\Downloads\EYcode\EYcode> venv\Scripts\activate
(venv) PS C:\Users\BN743PF\Downloads\EYcode\EYcode>
```

**Run the following command:**

***pip install -r requirements.txt***

You may see error during the installation mentioning something like below:

```
running bdist_wheel
running build
running build_ext
building 'hnsplib' extension
error: Microsoft Visual C++ 14.0 or greater is required. Get it with "Microsoft C++ Build Tools": https://visualstudio.mic
rosoft.com/visual-cpp-build-tools/
[end of output]

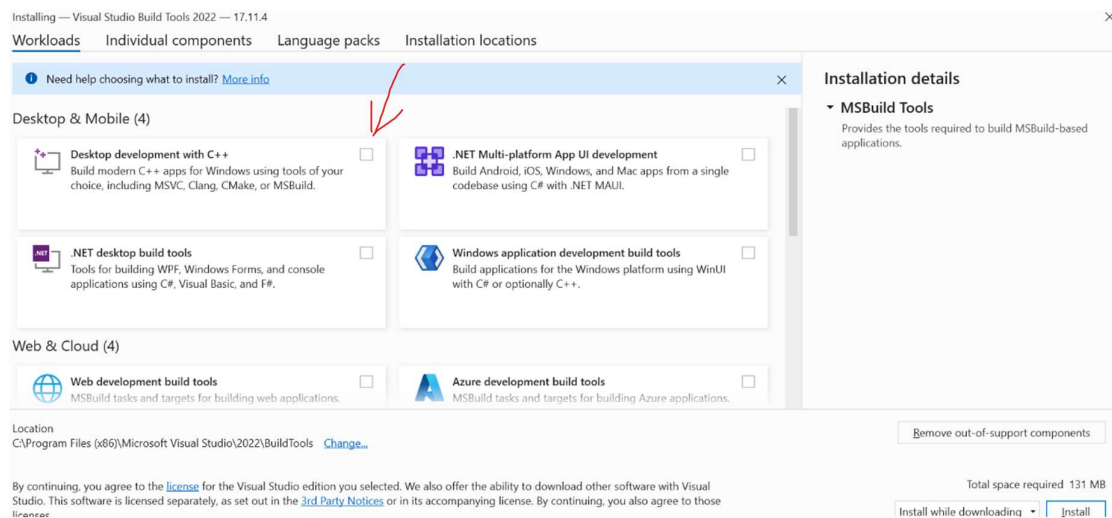
note: This error originates from a subprocess, and is likely not a problem with pip.
```

To fix it, install MS dev tools:

<https://visualstudio.microsoft.com/visual-cpp-build-tools/>

| Name                                                                                                              | Date modified       | Type                |
|-------------------------------------------------------------------------------------------------------------------|---------------------|---------------------|
| ▼ Today (5)                                                                                                       |                     |                     |
|  vs_BuildTools.exe               | 10-10-2024 02:32 PM | Application         |
|  tesla_db.zip                    | 10-10-2024 12:24 PM | Compressed (zipped) |
|  tesla-annual-reports.zip        | 10-10-2024 12:09 PM | Compressed (zipped) |
|  index_tesla_documents.ipynb     | 10-10-2024 12:00 PM | Jupyter Source File |
|  retrieval_tesla_documents.ipynb | 10-10-2024 11:53 AM | Jupyter Source File |
| ▼ Yesterday (6)                                                                                                   |                     |                     |

When you open the installer, select the first box below:



After installation, restart VS Code

And open the code folder.

**Reactivate the virtual environment** in the terminal:

In terminal, execute the following:

```
➤ venv\Scripts\activate
```

Install packages again:

```
pip install -r requirements.txt
```

## Troubleshooting

- If terminal says scripts are disabled (PowerShell):  
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
- If Python is not recognized, verify it's added to PATH
- Make sure to always activate virtual environment before installing packages. The active environment is shown in green as below:

Give command:

venv\Scripts\activate

The screenshot shows a VS Code terminal window with a Python script being executed. The script creates a twin axis plot for population and literacy rate. The output shows a table with city names and their corresponding population and literacy rates. Below the table, the terminal shows the command 'venv\Scripts\activate' being executed, which activates the virtual environment, indicated by the '(venv)' prompt.

```

45 # Create a twin axes for population and literacy rate
46 fig, ax1 = plt.subplots(figsize=(15, 8))

1   Delhi    29000000    86.2
2   Bangalore 13000000    88.7
3   Hyderabad 9700000    83.3
4   Ahmedabad 8000000    89.1

PS D:\code\EYcode>
* History restored

PS D:\code\EYcode>
* History restored

PS D:\code\EYcode> venv\Scripts\activate
(venv) PS D:\code\EYcode>

```

- If still facing errors, try following and then run *pip install -r requirements.txt*:  
pip install --upgrade  
pip setuptools wheel  
pip cache purge  
pip install --upgrade --force-reinstall setuptools

Ater successful installation of packages you should see:

The screenshot shows a VS Code terminal window with the output of a pip install command. The output shows the downloading and installation of various packages, including certifi, colorama, distro, h11, idna, jiter, openai, pydantic, and typing-extensions. Below the installation output, the terminal shows the command 'python -m venv venv' being executed, which creates a new virtual environment. The next command is 'venv\Scripts\activate', which activates the virtual environment, indicated by the '(venv)' prompt.

```

13 azure_config = {
14     "AZURE_OPENAI_ENDPOINT": os.getenv("AZURE_OPENAI_ENDPOINT"),
15     "AZURE_OPENAI_MODEL_DEPLOYMENT_NAME": os.getenv("AZURE_OPENAI_MODEL_DEPLOYMENT_NAME"),
16     "AZURE_OPENAI_MODEL_NAME": os.getenv("AZURE_OPENAI_MODEL_NAME"),
17     "AZURE_OPENAI_API_KEY": os.getenv("AZURE_OPENAI_API_KEY"),
18     "AZURE_OPENAI_API_VERSION": os.getenv("AZURE_OPENAI_API_VERSION")
19 }

Downloading certifi-2024.8.30-py3-none-any.whl (167 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading h11-0.14.0-py3-none-any.whl (58 kB)
Installing collected packages: typing-extensions, sniffio, jiter, idna, h11, distro, colorama, certifi, annot
ydantic-core, httpcore, anyio, pydantic, httpx, openai
Successfully installed annotated-types-0.7.0 anyio-4.6.0 certifi-2024.8.30 colorama-0.4.6 distro-1.9.0 h11-0.
httpx-0.27.2 idna-3.10 jiter-0.6.1 openai-1.51.2 pydantic-2.9.2 pydantic-core-2.23.4 sniffio-1.3.1 tqdm-4.66
s-4.12.2

PS C:\Users\BN743PF\Downloads\EYcode\EYcode> python -m venv venv
PS C:\Users\BN743PF\Downloads\EYcode\EYcode> venv\Scripts\activate
(venv) PS C:\Users\BN743PF\Downloads\EYcode\EYcode>

```

Thatsit!!

**You are now ready to build your GenAI Python apps with VS code**

## Python essentials

Download the python\_basicapps.zip from:

[https://github.com/vijay-agrawal/genaitraining/blob/main/code/00python\\_basicapps.zip](https://github.com/vijay-agrawal/genaitraining/blob/main/code/00python_basicapps.zip)

Unzip it into the **genaitraining** or your project root folder

It contains simple python scripts and a datafile **loanapproval\_hist\_data.csv**.

Open it in Excel and examine the data.

Loanapproved is the target column (value we want to teach the machine to predict) and other columns are features that influence the target.

## Using functions in python

The demo functions\_demo.py can be reviewed to understand how function calling, parameter passing is done in Python

## Working with data tables using pandas

<https://www.youtube.com/watch?v=tRKeLrwfUgU>

Review the pandas demo code present in pandas\_demo.py.

Run it by pressing the play button on top right.

Edit it to play with various dataframe capabilities such as sorting, adding a new derived column etc. Use help of your favourite AI chatbot to generate the code for the same and test it out.

## Using .env for API keys and private configuration

Best practice in python based application development is to keep private configuration parameters such as API Keys, passwords etc in a **.env** file

When your python application loads it can read the private key value pairs from the .env file and use them.

This gives flexibility and security and is a common practice to deal with private configuration parameters.

See the video below on how to setup a .env file for your project:

[https://www.youtube.com/watch?v=8dlQ\\_nDE7dQ](https://www.youtube.com/watch?v=8dlQ_nDE7dQ)

Open the dotenv\_demo.py in VS Code

Run it by using the “Play” button on top right in your VS Code.

Alternatively, you can open the terminal, activate your virtual environment



➤ Venv\scripts\activate

And issue following command:

➤ python dotenv\_demo.py

The script should run and print the environment variables present in your .env file

## Working with data files

Data files such as csv files can be easily loaded in Python. They are usually converted to pandas dataframe for further analysis.

Review the file reading demo code present in dataloading.py.

Run it by pressing the play button on top right.

You can check the data file it is loading, present in the data folder.

## Using matplotlib for data visualization and analysis

Exploratory Data Analytics (EDA) with python

Run the data\_visualization.py to see how matplotlib library helps with data visualization and analysis

A good video on how to do EDA with python:

<https://www.youtube.com/watch?v=xi0vhXFPegw>

Also search on Kaggle for existing notebooks people have made on the same.

<https://www.kaggle.com/code>

## Accessing and consuming services using APIs

Review and run the code weather\_api\_json.py

It uses Weather API service from openweathermap (You can register to get your own free key)

**Above should give a good handle on Python essentials for this program**

## Google Colab and Kaggle

Google colab and Kaggle (<https://kaggle.com>) provide a cloud based Python development environment using **Jupyter notebook** concept.

Notebook makes it easy to write and execute python code inline. Below are steps outlined for colab. You can do similar with Kaggle if colab is not workable for you. Kaggle also provides many pre-built notebooks by contributors that are helpful for learning.

## Create your first colab notebook

Get started with Google colab by going to:

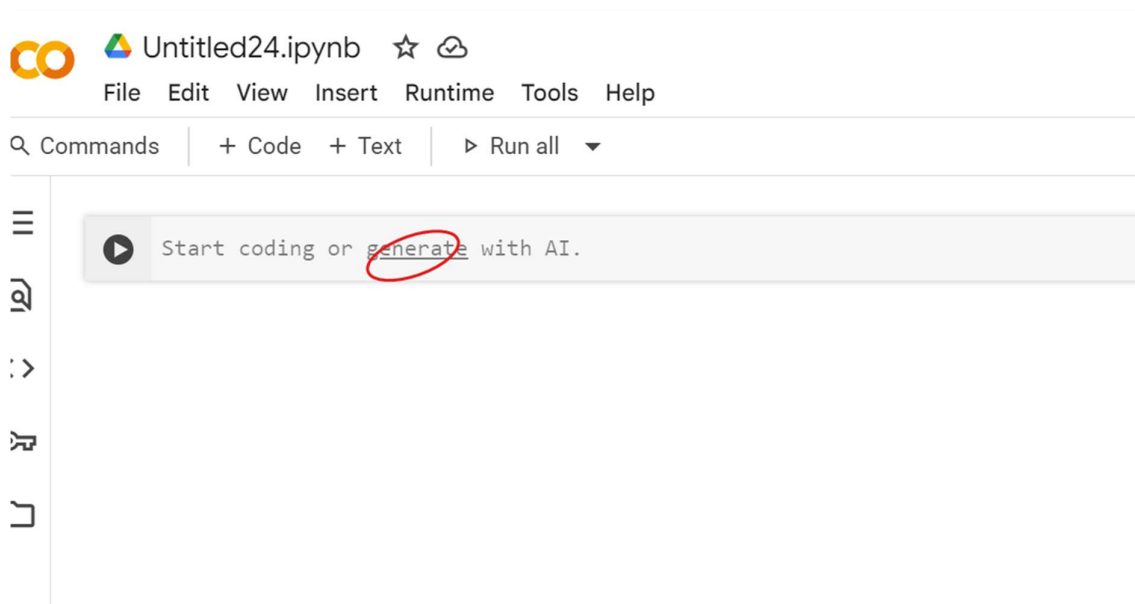
<https://colab.research.google.com/> and registering with your personal gmail account.

Create your first notebook by following the video at:

<https://www.youtube.com/watch?v=RLYoEylHL6A>

Try to put some of the python code from the python demos above into colab and execute them.

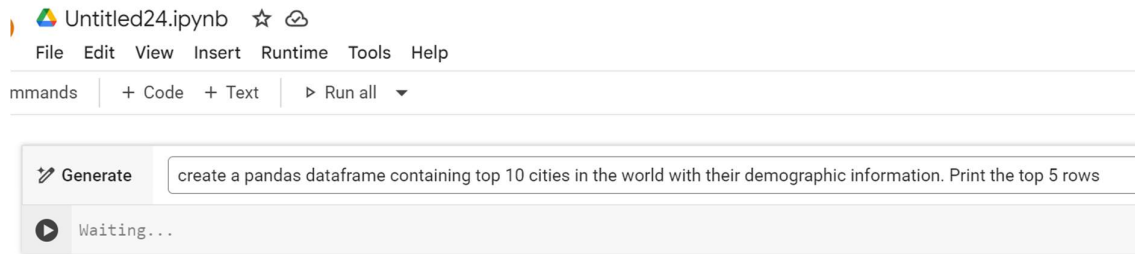
Google colab now also has **code generation** integrated into it!



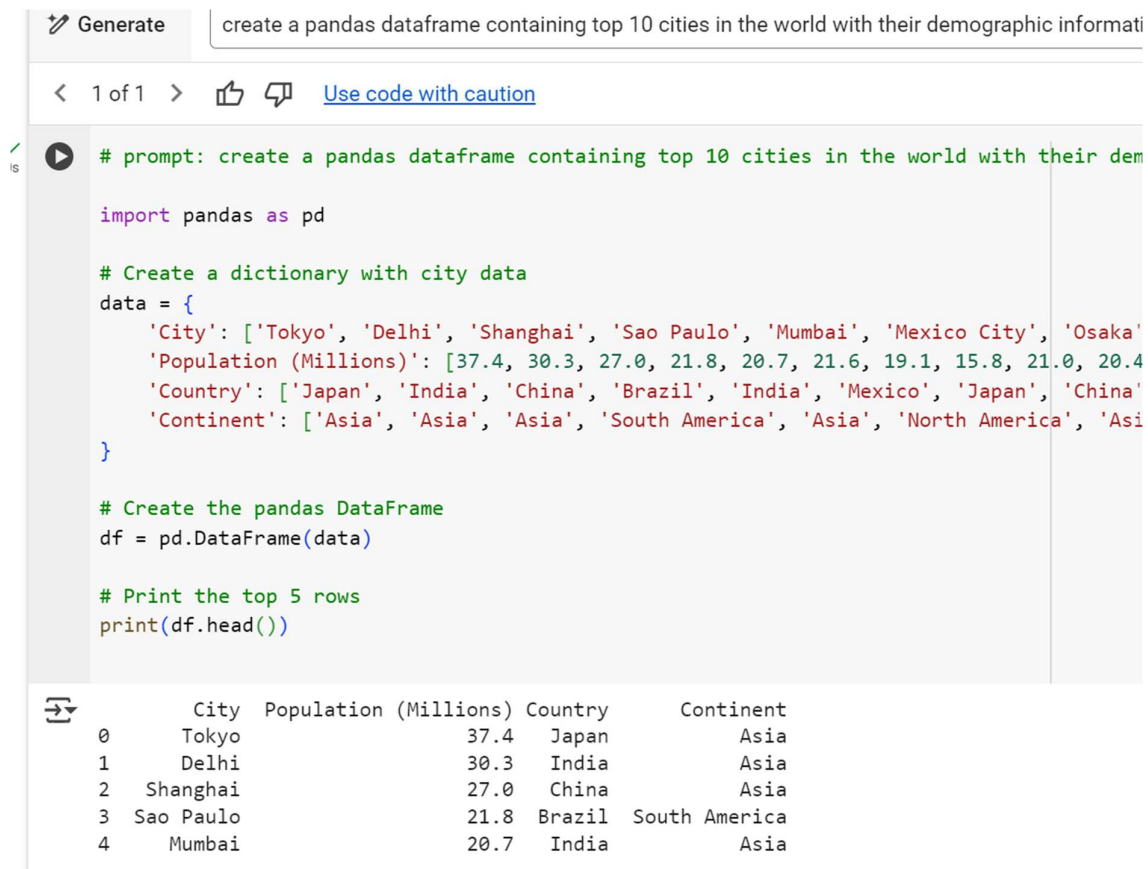
Click on “generate” and you can generate any code you want by just giving the prompt.

For example:

create a pandas dataframe containing top 10 cities in the world with their demographic information. Print the top 5 rows



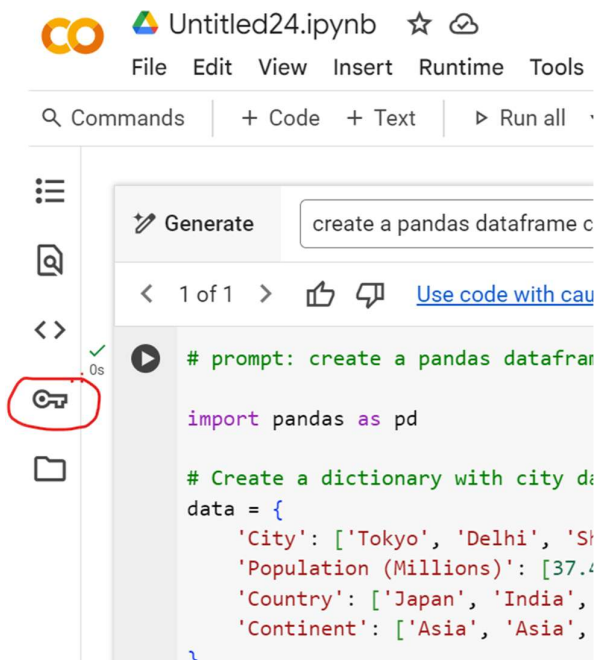
Click on the “Play” button and it will generate the code



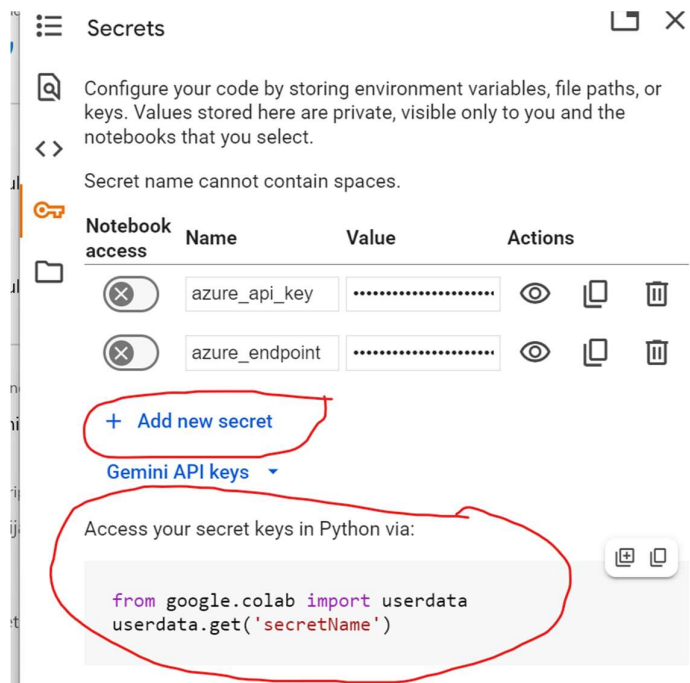
## Working with secret keys in Colab

Secret Keys are a way to keep your private configuration information such as API keys secure. (It is similar to the .env concept in your python project in VS Code)

Click on the “Key” icon:



You can add new KEY just like you did in .env file



It shows the code how to use them in your notebook

Copy that code and paste it in your notebook

## EXERCISE:

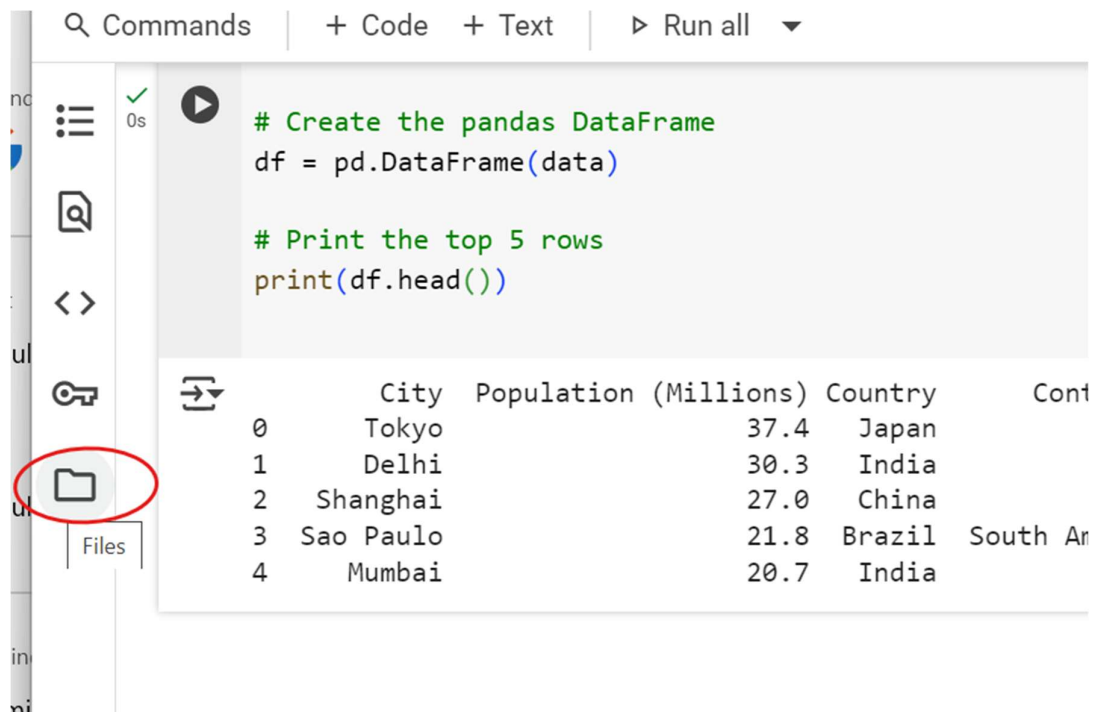
Take the code from the `weather_api_json.py` and make it work in colab.

Hint: remove the `load_dotenv()` and replace with `colabs' userdata.get()` after setting the `OPENWEATHERMAP_API_KEY` secret key in Colab

## Working with data files in Colab

Colab offers a filesystem where you can upload your data files and use them in the notebook.

Click on the folder icon on the left



The screenshot shows the Google Colab interface. At the top, there is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. Below this, a code cell is visible with the following Python code:

```
# Create the pandas DataFrame
df = pd.DataFrame(data)

# Print the top 5 rows
print(df.head())
```

Below the code cell, the output is displayed as a table:

|   | City      | Population (Millions) | Country | Cont     |
|---|-----------|-----------------------|---------|----------|
| 0 | Tokyo     | 37.4                  | Japan   |          |
| 1 | Delhi     | 30.3                  | India   |          |
| 2 | Shanghai  | 27.0                  | China   |          |
| 3 | Sao Paulo | 21.8                  | Brazil  | South Ar |
| 4 | Mumbai    | 20.7                  | India   |          |

On the left side of the interface, there is a sidebar with several icons. The folder icon, which is circled in red, is located below the code editor and above the 'Files' section.

Click on the “Upload” icon to upload a data file to session storage

The screenshot shows the Google Colab interface. At the top, the title bar says "Untitled24.ipynb" with a star and a cloud icon. Below it is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A toolbar contains "Commands", "+ Code", "+ Text", and "Run all". The left sidebar is titled "Files" and shows a file explorer. A red circle highlights the "Upload" icon (a document with an upward arrow). A tooltip "Upload to session storage" is visible. Below the upload icon, there is a folder named "sample\_data". The main area shows a code cell with the following code:

```
# Create the pandas DataFrame
df = pd.DataFrame(data)

# Print the top 5 rows
print(df.head())
```

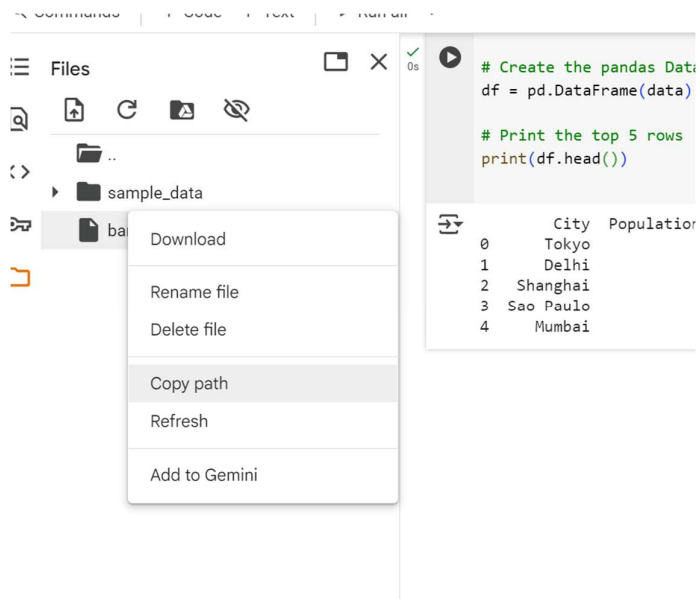
The code cell has a status bar showing a green checkmark and "0s". Below the code, the output is displayed as a table:

|   | City      | Populati |
|---|-----------|----------|
| 0 | Tokyo     |          |
| 1 | Delhi     |          |
| 2 | Shanghai  |          |
| 3 | Sao Paulo |          |
| 4 | Mumbai    |          |

Once the file is uploaded it will show up in the left pane.

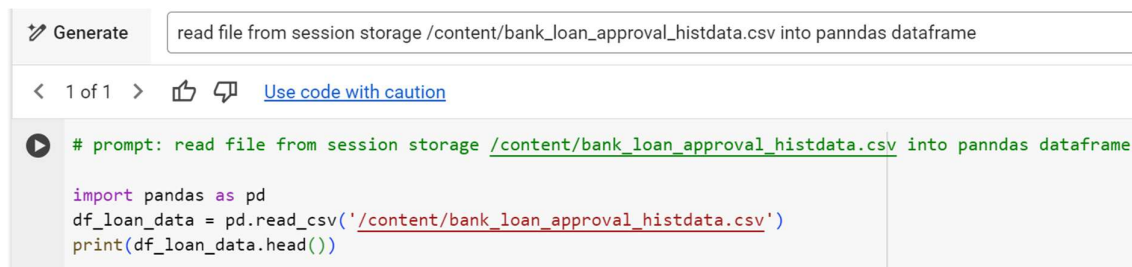
Click on the 3 dots, copy its path

The screenshot shows the "Files" sidebar in Google Colab. The file explorer shows a folder named "sample\_data" and a file named "bank\_loan\_approval\_histdata.c...". A red circle highlights the file "bank\_loan\_approval\_histdata.c...". The file has a status bar showing a green checkmark and "0s".



Move mouse toward center of the last cell to see the +Code button. Alternatively you can click on +Code in top level menu

Generate the code as below:



When you run the cell you will see the contents of the file printed

Watch videos below for more information:

Reading csv file:

<https://www.youtube.com/watch?v=VCllZKM7Njk>

How to upload files and use them in colab

<https://youtu.be/58t0PFIWR9Y?si=2oblRNYwyl5sk1Vq>

## Git (Optional)

Git is a version control tool and is needed when interacting your app with Huggingface (HF) Huggingface requires your code to be checked into a HF backed git repository

### 1. Download Git

1. Visit the official Git website: <https://git-scm.com/download/windows>
2. The download should start automatically for the latest version
3. If it doesn't start automatically, click on the download link for Windows

### 2. Install Git

1. Run the downloaded installer (git-X.XX.X-64-bit.exe)
2. Adjust your PATH environment:
  - Select "Git from the command line and also from 3rd-party software" (recommended)
3. Complete the installation
4. Note the Path in which git is installed. You may need to add it to the System environment variables if you cannot run git from command prompt

### 3. Configure Git

1. Open Command Prompt or Git Bash
2. Set your username and email:

```
git config --global user.name "Your Name"
git config --global user.email "your email"
```

**Note:** When copying pasting the above commands, sometimes spurious characters are getting introduced. Hence manually type these commands if copy paste is not working.

### 4. Verify Installation

1. Open Command Prompt or Git Bash
2. Type:

```
git --version
```
3. You should see the Git version number if installation was successful



## Troubleshooting

- If git commands aren't recognized, try restarting your command prompt or computer
- Ensure git is properly added to your PATH environment variable
- Check firewall settings if you have connectivity issues

For more detailed information and tutorials, visit the official Git documentation:

<https://git-scm.com/doc>

Following videos will help understand git better:

Git overview:

<https://www.youtube.com/watch?v=2ReR1YJrNOM>

Git process and commands:

[https://www.youtube.com/watch?v=e9lnsKot\\_SQ](https://www.youtube.com/watch?v=e9lnsKot_SQ)